

BERTopic for R: An Interface to the Python Implementation for Behavioral Research Workflows

Biying Zhou¹, Yuhao Yuan¹, Nanxin Deng², and Feng Ji¹

¹[TO DO: Affiliation]

²[TO DO: Affiliation]

[TO DO: Corresponding author name, postal address,
telephone number, and email]

Abstract

BERTopic provides an embedding-based approach to topic modeling, but its implementation and compatible machine-learning components are maintained primarily in Python. R users can connect to that ecosystem through general-purpose interoperability code or dedicated packages, although the resulting workflows still require explicit management of Python environments, object conversion, and model state. We describe **BERTopic**, an open-source R interface that retains the fitted Python model while exposing common operations through an S3 object, R functions, tibbles, and matrices. An SMS example illustrates environment selection, fitting, inspection, prediction, export, visualization, and persistence. We also tested whether the wrapper preserved selected outputs from equivalent direct Python code. Five paired fresh-process runs used the same 2,247 documents, frozen five-dimensional reduced embeddings, component objects, software versions, and process settings. The two paths returned identical assignments, metadata, and ordered term keys; weights and membership-strength matrices agreed within an absolute tolerance of 1e-12. In this controlled Windows task, paired R-

minus-Python differences had medians of 4.183 seconds for fitting, 11.400 seconds for cold-process time, and 200.0 MiB for peak memory. Because embedding generation and UMAP fitting were excluded, these measurements describe interface overhead rather than complete BERTopic performance. The contribution is an R-oriented access layer and a validation of its tested fitting and extraction path, not a new topic model or an independent implementation of BERTopic.

Keywords: topic modeling; BERTopic; R package; Python interoperability; behavioral text analysis; research software

1 Introduction

Behavioral researchers use open-ended survey responses, interviews, online discussions, clinical notes, and administrative records to study information that may be difficult to capture with closed-response measures. Exhaustive manual coding becomes costly as these collections grow. Topic models provide one way to summarize recurring patterns and construct document-level representations for subsequent qualitative inspection or quantitative analysis (Blei et al., 2003; Grimmer & Stewart, 2013; Roberts et al., 2014).

Classical methods such as latent semantic analysis, probabilistic latent semantic analysis, non-negative matrix factorization, and latent Dirichlet allocation represent documents primarily through word occurrence patterns (Blei et al., 2003; Deerwester et al., 1990; Hofmann, 1999; Lee & Seung, 1999). Transformer models instead provide contextual representations in which documents with different surface forms can occupy nearby regions of an embedding space (Devlin et al., 2019; Reimers & Gurevych, 2019; Vaswani et al., 2017). Embedding-based topic-modeling systems use this geometry to organize documents before deriving human-readable topic descriptions, which can be useful for short or lexically heterogeneous text (Angelov, 2020; Bianchi et al., 2021).

BERTopic is a modular framework in this family (Grootendorst, 2022). Its default

workflow combines contextual embeddings, UMAP dimensionality reduction (McInnes et al., 2018), HDBSCAN clustering (Campello et al., 2013; McInnes et al., 2017), and class-based TF-IDF (c-TF-IDF) topic representations. Researchers can replace these components, but the official implementation and the compatible embedding and clustering ecosystem are maintained primarily in Python.

This dependence on Python can complicate projects in which data management, statistical analysis, visualization, and reporting are conducted in R (R Core Team, 2026). R supports text-analysis workflows through `tm`, `tidytext`, and `quanteda` (Benoit et al., 2018; Feinerer et al., 2008; Silge & Robinson, 2016). Researchers can exchange files between R and Python or write `reticulate` code, although either route requires decisions about Python selection, argument passing, object conversion, and error diagnosis (Ushey et al., 2026). Dedicated R interfaces have also emerged. In particular, `bertopicr` provides helpers for Python-environment setup, model training, persistence, inspection, and visualization (Petrič, 2026). The relevant question is therefore not whether an R interface exists, but how a particular interface organizes the workflow, which backend versions it supports, and which operations have been tested.

The R interface does not change BERTopic’s statistical model. Its purpose is to call the official implementation from R and return selected results in forms that can be used in later R analyses. Evaluating whether the wrapper passes inputs and outputs correctly is separate from evaluating whether BERTopic produces substantively valid topics for a particular corpus.

We present `BERTopic`, an alternative R interface to the Python implementation. Its design emphasizes a retained Python model inside an S3 object, familiar methods such as `predict()`, `coef()`, `as.data.frame()`, and `fortify()`, explicit dense or sparse export, and functions for selecting and diagnosing the Python environment. A worked example demonstrates the intended workflow. We then compare the R call with equivalent direct Python code under a controlled configuration and report output agreement together with

descriptive runtime and memory measurements. This comparison examines the wrapper’s handling of selected inputs and outputs; it is not a comparison of two topic-modeling algorithms, a feature-completeness study of R packages, or a validation of the substantive topics recovered from the SMS corpus.

2 Computational Background and Scope

2.1 Scope of the R package

The Python library performs document embedding, dimensionality reduction, clustering, vectorization, topic weighting, and optional refinement of topic labels. The R package validates inputs, calls Python through `reticulate`, stores the fitted Python object, and converts selected results to R objects. It also provides functions for installing and selecting a Python environment, checking dependencies, and saving fitted models. The statistical properties of the fitted model remain those of the Python library and the component models chosen by the user.

The following summary describes the BERTopic pipeline only to show how the R functions correspond to the Python implementation. The worked example demonstrates use of the package. The later comparison checks selected results against a direct Python call and measures runtime and memory use. Neither exercise establishes that the topics from an unsupervised model are substantively valid.

2.2 The six-stage upstream pipeline

The R package delegates computation to the upstream Python library. The default BERTopic workflow can be summarized in six stages:

1. An `embedding_model` converts documents into dense contextual vectors.
2. A `umap_model` or another compatible reducer projects the vectors into a lower-

dimensional space.

3. An `hdbscan_model` or another compatible clusterer assigns documents to candidate topics and may identify outliers.
4. A `vectorizer_model` constructs topic-level term counts from the documents assigned to each cluster.
5. A `ctfidf_model` weights terms by their distinctiveness across topic-level document classes.
6. An optional `representation_model` refines, reranks, or relabels the topic representation.

The resulting object combines document assignments, topic-level term representations, fitted component objects, and optional membership-strength information. The outlier topic is conventionally labeled `-1`. BERTopic is not a fully generative word-count model, and its probability output should not be interpreted as an LDA-style posterior topic mixture without additional justification.

2.3 Methodological scope

The package does not reproduce UMAP, HDBSCAN, or c-TF-IDF in R. Their mathematical derivations are outside the scope of this paper and are available in the cited method papers and BERTopic documentation. Accordingly, the paper focuses on how arguments and objects are passed between R and Python, how Python environments are selected and checked, and whether selected outputs match those from a direct Python call.

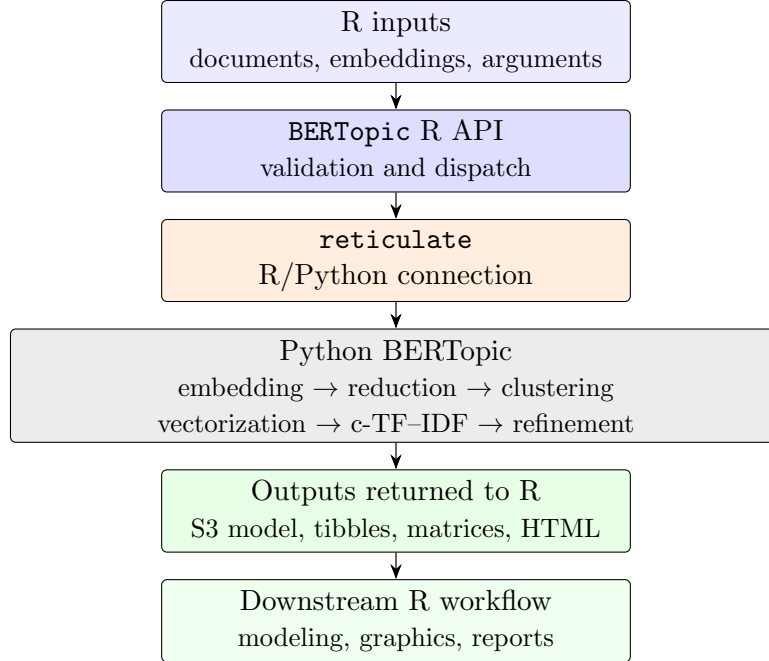


Figure 1: Architecture of the R interface. Core modeling is performed by the official Python library; the R package manages inputs, calls, conversion, and outputs returned to R.

3 Software Design and Implementation

3.1 Relationship to other R interfaces

The CRAN packages `BERTopic` and `bertopicr` were released in the same period and both connect R to the Python `BERTopic` ecosystem through `reticulate` (Petrič, 2026). Their functionality overlaps: both support environment setup, model fitting, inspection, persistence, and visualization. The present package is organized around a `bertopicr` S3 object that retains the upstream model and adds familiar R methods, explicit conversion to common R data structures, and environment diagnostics. These design choices provide an alternative API rather than an independent statistical implementation.

This article evaluates selected input and output mappings against direct Python execution; it does not provide a head-to-head usability study or establish that one R package is generally preferable. Researchers should consider backend-version support, the operations required for their analysis, the available tests, and the preferred object model when choosing

an interface. Table 1 therefore documents the scope of `BERTopic`; it should not be read as evidence that every listed helper has received the same level of validation.

3.2 R wrapper and model object

The main entry point, `bertopic_fit(text, embeddings = NULL, ...)`, calls `BERTopic` from an R session. The function validates that `text` is a character vector, checks that the Python library is available, imports the Python `bertopic` module, and forwards `...` to the Python `BERTopic()` constructor. If the user supplies a numeric R matrix of embeddings, the wrapper converts it to a Python-compatible array and passes it to `fit_transform()`; otherwise, the configured Python embedding model creates the embeddings.

The return value has S3 class `bertopic_r` and contains three main fields: `.py`, the fitted Python `BERTopic` object; `topics`, the document-level topic assignments returned during fitting; and `probs`, the membership-strength matrix when available. Storing `.py` gives users access to the complete fitted Python object. The wrapper converts common results to tibbles, R matrices, sparse matrices, or HTML files. Users can call the Python object directly for methods that do not yet have an R function.

Arguments supplied through `...` are passed to the Python constructor. Users may therefore create Python UMAP, HDBSCAN, vectorizer, c-TF-IDF, or representation objects with `reticulate` and pass them directly to `bertopic_fit()`. This exposes `BERTopic`'s configurable components within the backend versions explicitly supported by the R package. It does not guarantee forward compatibility with changed Python signatures. R also cannot validate every Python argument, so errors in advanced configurations may need to be diagnosed from the Python message and upstream documentation.

3.3 Functions and return values

Table 1 summarizes the principal R functions, the Python operation they invoke, and the result returned to R. The functions cover model fitting, inspection, prediction, visualization,

modification, and persistence.

Table 1: Mapping Between the R Interface and the Python BERTopic Backend

R function	Python operation	R result or purpose
<code>bertopic_fit()</code>	<code>BERTopic()</code> , <code>fit_transform()</code>	<code>bertopic_r</code> S3 object with retained Python model, assignments, and optional membership strengths
<code>bertopic_topics()</code>	<code>get_topic_info()</code>	Topic metadata as a tibble
<code>bertopic_topic_terms()</code>	<code>get_topic()</code>	Terms and c-TF-IDF weights as a tibble
<code>bertopic_find_topics()</code>	<code>find_topics()</code>	Query-nearest topic identifiers, scores, and labels
<code>bertopic_get_document_info()</code>	<code>get_document_info()</code>	Document-level metadata as a tibble
<code>bertopic_transform()</code>	<code>transform()</code>	New-document assignments and optional membership strengths
<code>bertopic_predict()</code>		
<code>bertopic_as_document_topic_matrix()</code>	Stored fitting output	Dense or sparse R matrix for downstream analysis
<code>bertopic_update_topics()</code>	<code>update_topics()</code>	Updated topic representations
<code>bertopic_visualize_*()</code>	Python visualization methods	Interactive HTML output
<code>bertopic_save()</code>	<code>save()</code> , <code>load()</code>	Persistent fitted Python model wrapped for R reuse
<code>bertopic_load()</code>		

The package also supplies S3 methods. `print()` and `summary()` provide compact model information; `predict()` presents transformation as a familiar prediction task; `coef()` returns top terms across topics; and `as.data.frame()` and `fortify()` support common R data workflows. These methods organize access to the fitted object but do not change its Python computations. Table 1 lists the main functions. The controlled comparison in Section 5 covers only the fitting and extraction functions used in that test, not every helper in the table.

3.4 Dependency management and diagnostics

Because the package calls Python, backend installation is part of normal use. Version 0.1.2 provides Conda and virtualenv helpers and `install_py_deps()` as a dispatcher. Both routes consume the same exact requirements and run the same import validation. The supported tested configuration is Windows, Python 3.10.21, and BERTopic 0.16.0. Full

resolved environments are retained with the outputs; other backend versions are outside this support claim.

The installation step is explicit rather than silently triggered by model fitting. Python machine-learning dependencies are large, may download model weights, and may require system-specific resolution. An explicit one-time operation makes this mutation visible. If the backend is missing, the fitting function stops with a diagnostic that names the active Python interpreter and the installation and binding calls required to continue.

Session binding is handled by `use_bertopic()`, `use_bertopic_condaenv()`, and `use_bertopic_virtualenv()`. If `reticulate` has already initialized a different interpreter, these functions stop and instruct the user to restart the R session. This behavior avoids a common failure mode in which package installation succeeds in one environment but the R session imports from another.

Two functions support routine diagnosis. `bertopic_available()` checks importability, while `bertopic_session_info()` reports the interpreter and exact versions for 11 key modules. Full Python distribution versions and R session information are also retained. The documented `bertopic_self_check()` fits, transforms, saves, loads, and compares cached fields, topic metadata, and post-load transformation outputs.

4 Worked Example

4.1 Dataset and example plan

This section demonstrates the main package functions using a 2,247-message subset of the UCI SMS Spam Collection included with the package (Almeida & Hidalgo, 2011). Each row contains a binary label and one English SMS message. The packaged subset contains all 747 spam messages and 1,500 of the 4,827 ham messages in the source collection. **[TODO: Document the rule and random seed, if any, used to select the 1,500 ham messages, and archive a script that reconstructs the packaged data from the UCI**

source.] The labels are not used to fit the unsupervised topic model; they are retained to show the data structure and to support optional downstream checks. The direct comparison with Python is reported separately in Section 5.

Table 2: Structure and Descriptive Summary of the Packaged `sms_spam` Data

Variable	R type	Role	Example or summary
<code>label</code>	Character	Reference label; not supplied to BERTopic	1,500 ham (66.8%); 747 spam (33.2%)
<code>text</code>	Character	One document per row; model input	“Nah I don’t think he goes to usf, he lives around here though”
Document length		Characters per message	Median = 84; IQR = 41–147; range = 2–790
Whitespace-delimited length		Descriptive word count	Median = 16; IQR = 8–26; range = 1–162

Note. Length statistics were computed before model fitting. The example text is reproduced from the packaged data.

The sequence covers environment diagnosis, fitting, inspection, semantic search, prediction, export, visualization, topic-representation updates, and persistence. It illustrates how the functions are intended to be combined; it is not a comparison of topic-modeling algorithms or evidence that topics recovered from this corpus represent behavioral constructs.

4.2 Environment check and fitting

The following code loads the data, creates the document vector, records the backend configuration, and explicitly fixes UMAP’s random state. Setting only the global R or NumPy seed is not sufficient to make every stochastic component deterministic.

```

1 library(BERTopic)
2 library(reticulate)

```

```

3
4 stopifnot(bertopic_available())
5 backend <- bertopic_session_info()
6
7 data("sms_spam", package = "BERTopic")
8 docs <- as.character(sms_spam$text)
9
10 set_bertopic_seed(42)
11 sentence_transformers <- reticulate::import("sentence_transformers",
12     convert = FALSE)
13 embedding_model <- sentence_transformers$SentenceTransformer(
14     "all-MiniLM-L6-v2", revision = "1110
15     a243fdf4706b3f48f1d95db1a4f5529b4d41"
16 )
17 umap <- reticulate::import("umap", convert = FALSE)
18 umap_model <- umap$UMAP(
19     n_neighbors = as.integer(15),
20     n_components = as.integer(5),
21     min_dist = 0,
22     metric = "cosine",
23     random_state = as.integer(42)
24 )
25
26 model <- bertopic_fit(
27     text = docs,
28     embedding_model = embedding_model,
29     umap_model = umap_model,
30     calculate_probabilities = TRUE
31 )

```

Listing 1: Checking the backend and fitting a BERTopic model from R.

The number and composition of topics depend on the embedding model, reducer,

clusterer, vectorizer, random state, and software versions. Users should therefore inspect the fitted object rather than expect the counts reported for the separate controlled evaluation below.

4.3 Topic inspection and search

```

1  topic_info <- bertopic_topics(model)
2  head(topic_info[c("Topic", "Count", "Name")], 8)
3
4  valid_topics <- topic_info$Topic[topic_info$Topic != -1]
5  bertopic_topic_terms(model, topic_id = valid_topics[1], top_n = 10)
6
7  bertopic_find_topics(
8    model,
9    query_text = "free subscription message",
10   top_n = 5
11 )
12
13 document_info <- bertopic_get_document_info(model, docs)
14 head(document_info, 8)

```

Listing 2: Inspecting and querying the fitted topic solution.

Topic terms, counts, and document-level information support human inspection, but labels remain interpretations of cluster-specific patterns. They are not automatically validated behavioral constructs. Researchers should examine multiple documents per topic and document any coding or adjudication procedure used to assign substantive labels.

4.4 Prediction and R-native export

```

1  new_docs <- c(
2    "Love you so much, see you tonight.",
3    "Free subscription! Reply STOP to unsubscribe."

```

```

4 )
5
6 new_predictions <- predict(model, new_docs, type = "both")
7 new_predictions$topics
8 dim(as.matrix(new_predictions$probs))
9
10 document_topic <- bertopic_as_document_topic_matrix(
11   model,
12   sparse = FALSE,
13   prefix = TRUE
14 )
15 stopifnot(nrow(document_topic) == length(docs))

```

Listing 3: Transforming new documents and exporting training-corpus output.

When `calculate_probabilities = TRUE`, the matrix contains HDBSCAN-derived membership strengths for the non-outlier topics supported by the fitted model. These values can be used descriptively or as features in downstream models, but their measurement interpretation and uncertainty should be evaluated for the substantive application.

4.5 Visualization, representation updates, and persistence

The wrapper writes Python Plotly figures to HTML, which preserves hover information and can be shared as supplementary material. Version 0.1.2 writes HTML that references the Plotly content-delivery network, so the archive should also contain static captures or self-contained exports for offline inspection.

```

1 bertopic_visualize_topics(
2   model,
3   file = "viz_topics.html"
4 )
5
6 bertopic_visualize_heatmap(

```

```

7     model,
8     file = "viz_heatmap.html"
9 )
10
11 # Topic representations can be recomputed after inspection.
12 bertopic_update_topics(model, docs)
13 updated_topic_info <- bertopic_topics(model)
14
15 # Save the fitted Python model; load only trusted serialized models.
16 bertopic_save(
17     model,
18     path = "sms_bertopic_model",
19     serialization = "safetensors",
20     overwrite = TRUE
21 )
22 restored_model <- bertopic_load("sms_bertopic_model")
23 restored_topic_info <- bertopic_topics(restored_model)
24 stopifnot(identical(
25     as.data.frame(restored_topic_info)[c("Topic", "Count", "Name")],
26     as.data.frame(updated_topic_info)[c("Topic", "Count", "Name")]
27 ))

```

Listing 4: Creating HTML visualizations and saving the fitted model.

The complete worked example was rerun using the installed corrected release and pinned model revision. Its printed output, tables, HTML, Plotly JSON, and static figures come from its own fitted model. The safetensors check verifies topic metadata; the additional full pickle round trip verifies cached assignments/strengths and post-load predictions. Lightweight formats omit the reducer and clusterer, so the metadata check does not assert identical HDBSCAN inference after lightweight loading.

The SMS data contain no genuine time variable. We therefore do not interpret a topics-over-time analysis for this corpus. The functions `bertopic_topics_over_time()`

and `bertopic_visualize_topics_over_time()` are available for datasets with substantively meaningful timestamps.

5 Comparison with the Python Implementation

5.1 Benchmark design

Because the R and Python calls use the same Python library, a comparison of topic quality would not compare two different algorithms. We instead tested whether the R wrapper returned the same selected outputs as equivalent Python code and measured the additional runtime and memory used by the R process. This test does not assess whether BERTopic discovers substantively valid topics.

The benchmark used 2,247 SMS messages and installed **BERTopic** 0.1.2, tag v0.1.2, source commit faee106. The exact source archive checksum is retained with the outputs. Embeddings were generated once with all-MiniLM-L6-v2 at immutable revision 1110a243fdf4706b3f48f1d95db1a4f5529b4d41, then reduced from 384 to five dimensions with UMAP and `random_state=42`. The frozen array was stored as float64 in column-major order to match R’s conversion through **reticulate**. Both paths used `BaseDimensionalityReduction()`, HDBSCAN with `min_cluster_size=10` and `prediction_data=TRUE`, an English-stop-word `CountVectorizer`, and `calculate_probabilities=TRUE`. Document and array SHA-256 hashes were 2f90a92a...fb036675 and f8169618...92bc3531; full hashes and preparation settings are retained in the manifest.

Five fresh Python processes and five fresh R processes ran on Windows x64 (build 26200), with 8 physical cores, 16 logical processors, and 15.9 GiB of physical memory visible to the operating system. The stack was R 4.4.1, Python 3.10.21, BERTopic 0.16.0, NumPy 1.26.4, scikit-learn 1.4.2, PyTorch 2.1.2+cpu, transformers 4.47.0, UMAP 0.5.5, and HDBSCAN 0.8.37. Process order alternated, with R first on odd repetitions. Both paths inherited `PYTHONHASHSEED=42` and identical single-thread settings for OpenMP

and common numerical libraries. Components were constructed before timing. Python used `time.perf_counter()`; R used elapsed `proc.time()`. R fit timing includes wrapper validation and conversion. Cold-process time includes startup, imports, input loading, component construction, fitting, and output writing. Windows peak resident memory was read from `psutil.Process().memory_info().peak_wset`. These are workflow measurements, not a language-neutral microbenchmark.

5.2 Comparison of model outputs

The controlled model produced 50 non-outlier topics and 808 outlier messages (36.0%). Table 3 shows the eight largest topics. Terms are model outputs rather than author-assigned labels and do not validate behavioral constructs.

Table 3: Largest Non-Outlier Topics in the Controlled SMS Model

Topic	Messages	Four highest-weight terms
0	122	love, life, miss, smile
1	94	chat, text, stop, sex
2	80	tone, ringtone, tones, 8007
3	64	pub, drive, car, im
4	52	class, exam, school, teaches
5	48	won, draw, claim, prize
6	46	carlos, hes, says, tyler
7	46	unsubscribe, free, sms, goto

Note. Terms and counts are reproduced from the frozen-input benchmark output. Topic identifiers are arbitrary model labels.

Table 4 reports the paired comparison. All five pairs returned matching 2,247 assignments, 808 outliers, `Topic/Count/Name` metadata in 51 rows, and ordered top-ten term keys. Maximum absolute weight and membership-strength differences were $5e-16$ and $4.86e-16$, below the 10^{-12} tolerance. Membership-strength matrices had dimensions 2247×50 .

Under this configuration, the R wrapper and direct Python code produced the same selected outputs. The comparison does not imply that separate stochastic fits will always be identical, nor does it validate the substantive interpretation of the recovered topics.

Table 4: Comparison of R and Python Outputs Across Five Paired Runs

Output	Direct Python	R interface	Comparison
Document assignments	2247	2247	Exact in 5/5 pairs
Outlier assignments	808	808	Exact
Topic-information rows	51	51	Exact
Non-outlier topics	50	50	Exact
Ordered top-term keys	510	510	Exact
Maximum c-TF-IDF difference	—	—	5e-16
Membership-strength dimensions	2247×50	2247×50	Exact
Maximum probability difference	—	—	4.86e-16

5.3 Runtime and memory use

Table 5 reports medians and interquartile ranges from five fresh processes per interface. Median fitting took 0.630 seconds in Python and 4.810 seconds in R. Paired R-minus-Python differences had medians of 4.183 seconds for fitting, 11.400 seconds for cold-process time, and 200.0 MiB for peak memory. Paired-difference medians are summarized separately from medians for each interface.

Table 5: Runtime and Peak-Memory Benchmark

Interface	Fit time, s	Cold-process time, s	Peak RSS, MiB
Direct Python	0.630 [0.630, 0.633]	16.950 [16.870, 16.960]	402.2 [402.1, 402.4]
R interface	4.810 [4.810, 4.830]	28.300 [28.280, 28.410]	602.9 [601.3, 604.9]

Note. Values are medians [interquartile ranges] from five fresh processes. Embedding generation and UMAP reduction were performed once before the timed paired calls. RSS = resident set size.

Direct Python used less time and memory in this Windows configuration. Paired median differences were 4.183 seconds for fitting, 11.400 seconds for cold-process time, and 200.0 MiB for peak memory. Embedding generation and reduction were excluded. These five replications of one corpus and configuration do not establish general performance advantages.

5.4 Reproducibility under stochastic fitting

Matching results in the controlled comparison and reproducibility across independent model fits are different questions. UMAP, approximate neighbor search, transformer execution, and hardware-specific numerical libraries may introduce variation. Topic identifiers can also change when clusters are reordered, even if their content is similar. For reproducible analyses, users should (a) set R and Python seeds; (b) pass `random.state` explicitly to stochastic components that expose it; (c) record the R and Python package versions and the selected interpreter; (d) save the fitted model; and (e) evaluate topic stability across seeds or reasonable specifications when substantive conclusions depend on the discovered topics. Comparisons across stochastic fits should align topics by their documents or representations rather than assuming that equal numeric identifiers denote equal topics.

6 Availability, Installation, and Operational Guidance

6.1 Software availability

The evaluation used the locally versioned `BERTopic` 0.1.2 source archive. Development source and issue tracking are at <https://github.com/Feng-Ji-Lab/BERTopic>; the CRAN project page is <https://CRAN.R-project.org/package=BERTopic>. The exact tested source and checksum are retained with the analysis materials. Public versioned release publication and an archive DOI remain pending. The R interface is MIT-licensed; dependencies retain their respective licenses.

6.2 One-time backend setup

```

1  install.packages("provenance/releases/BERTopic_0.1.2.tar.gz",
2                      repos = NULL, type = "source")
3  library(BERTopic)
4

```

```

5 # One-time environment creation and dependency installation.
6 # An explicit route avoids an unrecorded fallback.
7 install_py_deps_conda(
8     envname = "r-bertopic",
9     python_version = "3.10",
10    validate = TRUE
11 )
12
13 # Restart R before binding if reticulate initialized another Python.
14 use_bertopic_condaenv("r-bertopic")
15
16 stopifnot(bertopic_available())
17 bertopic_session_info()

```

Listing 5: Installing and binding the Python backend.

The environment creation step can require several minutes and network access. A sentence-transformer model may also be downloaded on first use. Version 0.1.2 also provides a virtualenv route, but either route must be validated against its fully resolved dependency set. Production analyses should preserve that specification rather than assuming that a later installation will select the same transitive versions.

6.3 Common installation and runtime problems

7 Discussion

The controlled comparison establishes a limited but useful result: for the frozen inputs and core fitting and extraction path tested here, the R wrapper preserved the selected outputs of direct Python execution. The additional time and memory reflect the cost of running R together with an embedded Python interpreter, but the short controlled task is not representative of an end-to-end analysis that includes transformer inference and UMAP.

Table 6: Common Installation and Runtime Problems

Observed problem	Likely cause	Recommended action
<code>bertopic_available()</code> is false	BERTopic is absent from the active interpreter	Run <code>bertopic_session_info()</code> , verify the Python path, then run <code>install_py_deps()</code>
Requested environment differs from active Python	<code>reticulate</code> initialized too early	Restart R, call <code>use_bertopic()</code> before importing another Python module
Transformer model cannot be downloaded	Network, cache, or model-hub access	Pre-download the model in the bound environment or supply precomputed embeddings
Model fits but results differ across runs	A stochastic component lacks an explicit state	Pass <code>random_state</code> to UMAP or other relevant components and record all versions
Saved model cannot be restored elsewhere	Backend or serialization mismatch	Record versions, prefer a supported portable serialization, and test restoration before archiving

The evidence therefore supports an input–output mapping claim, not a general claim about computational efficiency, topic quality, or the reliability of every exported helper.

The practical value of the interface lies in keeping the fitted Python model and its commonly used outputs inside an R project. Researchers can move topic assignments, terms, membership strengths, and visualizations into later R analyses without repeatedly exchanging intermediate files. This benefit is most relevant when data preparation, statistical modeling, graphics, and reporting are already conducted in R. Direct Python remains the simpler option for Python-centered projects, for immediate access to newly released upstream features, or when avoiding the memory cost of two runtimes is important. Researchers who prefer an R interface should also compare the object model, backend support, documentation, and tests of `BERTopic` with those of `bertopicr`; the present study does not establish the superiority of either package.

Cross-language convenience does not remove the need for software-version control. Python package resolution, compiled libraries, model downloads, and hardware differences can prevent the same code from running elsewhere. The use of `...` exposes advanced constructor arguments but cannot protect users from changed upstream signatures or incompatible component versions. The tested backend is pinned to Python `BERTopic` 0.16.0. A citable analysis should therefore record the exact R and Python environments, save component settings, verify model restoration, and archive the package source used for the results. Organizing research products as versioned R packages and preserving the associated materials can make those dependencies more visible (Vuorre & Crump, 2021).

The controlled benchmark covers one Windows configuration, one short-text corpus, one backend version, and five timing replications; embedding generation and reduction are excluded. The corrected Windows package passed its full regression suite without failures, warnings, or skips. This evidence does not validate every constructor configuration or establish substantive topic quality.

The substantive limits of unsupervised topic modeling are unchanged by the interface.

A coherent list of top words does not establish that a topic is a reliable psychological construct. Short documents near cluster boundaries may move between a substantive topic and the outlier class under modest specification changes, and HDBSCAN-derived membership strengths are not automatically calibrated probabilities of theoretically defined categories. Behavioral applications should justify the unit of analysis and embedding model, report preprocessing and component choices, inspect multiple representative documents, assess stability across reasonable specifications, and validate any topic-derived measure against its intended use. Evaluations of behavioral text-mining methods similarly emphasize that convenient model output cannot substitute for task-specific validation against human judgments or other relevant criteria (Macanovic & Przepiorka, 2024).

In its current form, `BERTopic` provides an R-oriented access layer and preserves the selected outputs tested here. The contribution is interface design, workflow documentation, and bounded input–output validation. **[TO DO: Before submission, publish the tested versioned release, supply the archive DOI, and complete outstanding dataset-provenance and author-information fields.]**

Declarations

Funding. **[TO DO: Insert complete funding statement or “No funding was received for conducting this study.”]**

Competing interests. **[TO DO: Insert author-approved competing-interests statement.]**

Ethics approval. Not applicable. The software demonstration analyzes an existing public dataset distributed with the package.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

Availability of data and materials. The demonstration data are distributed in

the `BERTopic` R package. **[TO DO: Insert the archival OSF/Zenodo URL containing the exact benchmark inputs or a deterministic generation script, outputs, figures, and manuscript materials.]**

Code availability. Development source is at <https://github.com/Feng-Ji-Lab/BERTopic>. The benchmark used version 0.1.2, tag v0.1.2, commit faee106, and the retained source archive checksum. **[TO DO: Insert a public versioned release and DOI for the exact source and analysis scripts.]**

Author contributions. **[TO DO: Complete an author-approved CRediT contribution statement. Ensure that manuscript authorship and package-development contributions are documented consistently.]**

Open Practices Statement

The R package, source code, documentation, packaged demonstration data, benchmark scripts, raw benchmark outputs, and manuscript source will be available in a versioned public archive at **[TO DO: OSF or Zenodo URL/DOI]**. The software evaluation was not preregistered.

References

- Almeida, T., & Hidalgo, J. (2011). SMS spam collection. <https://doi.org/10.24432/C5CC84>
- Angelov, D. (2020). Top2Vec: Distributed representations of topics. <https://doi.org/10.48550/arXiv.2008.09470>
- Benoit, K., Watanabe, K., Wang, H., Nulty, P., Obeng, A., Müller, S., & Matsuo, A. (2018). quanteda: An R package for the quantitative analysis of textual data. *Journal of Open Source Software*, 3(30), 774. <https://doi.org/10.21105/joss.00774>
- Bianchi, F., Terragni, S., & Hovy, D. (2021). Pre-training is a hot topic: Contextualized document embeddings improve topic coherence. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint*

- Conference on Natural Language Processing (Volume 2: Short Papers)*, 759–766. <https://doi.org/10.18653/v1/2021.acl-short.96>
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent dirichlet allocation. *Journal of Machine Learning Research*, 3(Jan), 993–1022. <https://www.jmlr.org/papers/v3/blei03a.html>
- Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in knowledge discovery and data mining* (pp. 160–172). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-37456-2_14
- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 391–407. [https://doi.org/10.1002/\(SICI\)1097-4571\(199009\)41:6<391::AID-ASII1>3.0.CO;2-9](https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASII1>3.0.CO;2-9)
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Feinerer, I., Hornik, K., & Meyer, D. (2008). Text mining infrastructure in R. *Journal of Statistical Software*, 25(5), 1–54. <https://doi.org/10.18637/jss.v025.i05>
- Grimmer, J., & Stewart, B. M. (2013). Text as data: The promise and pitfalls of automatic content analysis methods for political texts. *Political Analysis*, 21(3), 267–297. <https://doi.org/10.1093/pan/mps028>
- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. <https://doi.org/10.48550/arXiv.2203.05794>
- Hofmann, T. (1999). Probabilistic latent semantic indexing. *Proceedings of the 22nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 50–57. <https://doi.org/10.1145/312624.312649>

- Lee, D. D., & Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. *Nature*, *401*(6755), 788–791. <https://doi.org/10.1038/44565>
- Macanovic, A., & Przepiorka, W. (2024). A systematic evaluation of text mining methods for short texts: Mapping individuals’ internal states from online posts. *Behavior Research Methods*, *56*, 2782–2803. <https://doi.org/10.3758/s13428-024-02381-9>
- McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, *2*(11), 205. <https://doi.org/10.21105/joss.00205>
- McInnes, L., Healy, J., Saul, N., & Großberger, L. (2018). UMAP: Uniform manifold approximation and projection. *Journal of Open Source Software*, *3*(29), 861. <https://doi.org/10.21105/joss.00861>
- Petrič, T. (2026). *bertopicr: Topic modeling with BERTopic* [R package version 0.3.6]. <https://doi.org/10.32614/CRAN.package.bertopicr>
- R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://doi.org/10.32614/R.manuals>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Roberts, M. E., Stewart, B. M., Tingley, D., Lucas, C., Leder-Luis, J., Gadarian, S. K., Albertson, B., & Rand, D. G. (2014). Structural topic models for open-ended survey responses. *American Journal of Political Science*, *58*(4), 1064–1082. <https://doi.org/10.1111/ajps.12103>
- Silge, J., & Robinson, D. (2016). tidytext: Text mining and analysis using tidy data principles in R. *Journal of Open Source Software*, *1*(3), 37. <https://doi.org/10.21105/joss.00037>
- Ushey, K., Allaire, J. J., & Tang, Y. (2026). *reticulate: Interface to Python* [R package version 1.46.0]. <https://doi.org/10.32614/CRAN.package.reticulate>

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- Vuorre, M., & Crump, M. J. C. (2021). Sharing and organizing research products as R packages. *Behavior Research Methods*, 53, 792–802. <https://doi.org/10.3758/s13428-020-01436-x>